BrainNet viability as a Python backend for cortical reconstruction

BrainNet is a viable but immature integration target. It delivers **~50% better cortical thickness accuracy than the only comparable tool** (recon-all-clinical), arXiv +3 runs in ~1 second on GPU with CPU fallback, arXiv outputs per-hemisphere GIfTI surfaces, and requires no FreeSurfer license. The critical blocker: **it has no documented public Python API** — only installation instructions exist in the README, and you'll need to reverse-engineer the source code or go through the full SimNIBS CHARM pipeline. For `robust_laplacian`, grid-structured voxel coordinates are explicitly warned against; marching cubes first, then `mesh_laplacian()` is the correct path.

BrainNet is standalone, not pip-installable, and FreeSurfer-free

BrainNet lives at `github.com/simnibs/brainnet` Springer (GPL-3.0, GitHub github v0.2, Aug 2025, GitHub 139 commits, 4 stars). GitHub It is **not on PyPI** — the "brainnet" package on PyPI is an unrelated 2019 AI library. PyPI It is also **not bundled with SimNIBS** v4.5; the SimNIBS docs don't mention it. No FreeSurfer license is required at inference time (FreeSurfer was used only for generating training ground truth).

Installation requires cloning the repo and manually building CUDA extensions: GitHub

```bash
# 1. Install PyTorch + Kaolin
pip install torch==2.6.0 torchvision==0.21.0 --index-url https://download.py
pip install kaolin==0.18.0 -f https://nvidia-kaolin.s3.us-east-2.amazonaws.c

# 2. Clone and install
git clone https://github.com/simnibs/brainnet.git
cd brainnet
pip install -e .

# 3. Build CUDA extensions manually
conda install -c conda-forge cudatoolkit-dev=12.4
cd brainnet/mesh/cuda
python build.py build_ext --inplace
```

The hard dependencies are **PyTorch 2.6.0**, **NVIDIA Kaolin 0.18.0**, and **CUDA 12.4**. The Kaolin dependency is particularly heavy — it's NVIDIA's 3D deep learning library and pins you to specific torch/CUDA version combinations. A `pyproject.toml` and `environment.yaml` exist in the repo but their exact contents aren't publicly rendered on GitHub.

The standalone Python API is undocumented — CHARM is the only documented path

This is the most significant finding for integration viability. **The BrainNet GitHub README contains zero usage examples, no API documentation, no tutorials, and no Jupyter notebooks.** The only documented way to invoke BrainNet is through SimNIBS's CHARM pipeline:

```bash
# CLI (requires full SimNIBS)
charm ernie org/ernie_T1.nii.gz
```

```python
# Python (requires full SimNIBS)
from simnibs.segmentation import charm_main
charm_main.run(subject_dir="m2m_ernie", T1="ernie_T1.nii.gz", create_surface
```

Inside CHARM, the call chain is `charm_main.py` → `brain_surface` module → BrainNet neural network. The `brain_surface` module handles the actual mesh deformation inference. GitHub The `tools/` directory in the BrainNet repo likely contains utility scripts, but its contents aren't accessible without cloning the repo.

For integration as a backend, you would need to: clone the repo, read the source code in `brainnet/`, identify the model loading and inference functions, and call them directly. The architecture is a Graph Convolutional Network (UNet-like, 4 encoder levels with 64 channels each, 3 decoder levels) that deforms a template mesh arXiv through **6 refinement levels** to produce **245,762 vertices** per hemisphere. It first deforms to the white matter surface, then applies a simpler linear deformation to estimate the pial surface. Springer +2

Output format: per-hemisphere GIfTI with 245K vertices

BrainNet produces **separate left and right hemisphere** surfaces in **GIfTI (.gii) format**, stored in `m2m_subID/surfaces/`: Simnibs

- `lh.white.gii` / `rh.white.gii` — White matter surfaces
- `lh.pial.gii` / `rh.pial.gii` — Pial (gray matter) surfaces
- `lh.central.gii` / `rh.central.gii` — Mid-cortical surfaces (computed from WM + pial)
- `lh.sphere.reg.gii` / `rh.sphere.reg.gii` — Spherical registrations to FsAverage

The CHARM pipeline writes these to disk via `write_gifti_surface()`. GitHub The SimNIBS `Msh()` object exposes numpy arrays:

```python
import simnibs
msh = simnibs.read_gifti_surface("m2m_ernie/surfaces/lh.pial.gii")
vertices = msh.nodes.node_coord   # Nx3 numpy array
faces = msh.elm.node_number_list  # Mx3 numpy array
```

Cortical thickness is derivable from the WM-to-pial vertex correspondence (the paper validates thickness accuracy extensively). **Curvature and sulcal depth** overlays are not explicitly mentioned in the BrainNet output, though they could be computed post-hoc from the surfaces. Surface field overlays in SimNIBS use FreeSurfer `.curv` format. Simnibs

GPU: CUDA 12.4 required, CPU works as fallback, VRAM unspecified

The paper states processing time is **~1 second on GPU** and **a few minutes on CPU**. arXiv CPU fallback is explicitly supported — a major advantage for broader usability.

Requirement	Detail
CUDA version	12.4 (pinned by PyTorch wheel)
VRAM	Not published; estimate **4–8 GB** based on architecture (volumetric encoder + 245K-vertex GCN decoder + Kaolin mesh ops)
CPU fallback	Yes, confirmed in paper
Custom CUDA kernels	Required, manual build step in `brainnet/mesh/cuda/` GitHub

The custom CUDA extension and Kaolin dependency mean the GPU path is **not trivially portable** — you're locked to NVIDIA GPUs with CUDA 12.4 compatibility. The CPU path likely bypasses the custom CUDA kernels but still needs PyTorch.

Accuracy: substantially better than recon-all-clinical, approaching FreeSurfer quality

Benchmarked on 200 ADNI subjects (axial FLAIR, 0.85×0.85×5 mm^3) and 1,332 clinical subjects (MGH, mixed contrasts/resolutions), using FreeSurfer on paired 1mm isotropic T1w scans as ground truth: arXiv ResearchGate

Metric	BrainNet	RAC (recon-all-clinical)
Cortical thickness error	**0.24 mm**	0.50 mm
Mean surface distance (WM)	**0.800 mm**	0.940 mm
Mean surface distance (pial)	**0.796 mm**	0.926 mm
90th %ile Hausdorff (WM)	**1.942 mm**	2.376 mm
90th %ile Hausdorff (pial)	**2.049 mm**	2.689 mm

On pathological brains: no published evaluation exists. The clinical dataset from MGH likely includes some pathological cases incidentally, but the paper does not stratify results by pathology. The synthetic domain-randomization training strategy provides some theoretical robustness to unexpected anatomy, arXiv but this is unvalidated. SimNIBS's own documentation warns that its older CAT12-based cortical reconstruction "is not designed to work with pathologies" Springer — BrainNet may be more robust but there is no evidence yet.

BrainNet and arXiv 2505.14017 are the same project

The paper "End-to-End Cortical Surface Reconstruction from Clinical Magnetic Resonance Images" (arXiv:2505.14017) **is the BrainNet paper**. Same authors (Nielsen, Gopinath, Hoopes, Dalca, Magdamo, Arnold, Das, Thielscher, Iglesias, Puonti), ResearchGate same code repository. arXiv Published at **MLMI 2025** (MICCAI workshop), Springer LNCS vol. 16241, Springer pp. 212–223, online January 2026. springer The paper explicitly states: "The code is publicly available at https://github.com/simnibs/brainnet." Springer arXiv

Grid-structured point clouds will break robust_laplacian — use marching cubes instead

The `robust_laplacian.point_cloud_laplacian()` function (from `nmwsharp/robust-laplacians-py`, Sharp & Crane, SGP 2020 Best Paper) (github) has this exact signature:

```python
L, M = robust_laplacian.point_cloud_laplacian(
    points,              # Vx3 numpy float64
    mollify_factor=1e-5, # intrinsic mollification amount
    n_neighbors=30       # KNN for local Delaunay triangulation
)
# L: NxN positive semi-definite Laplacian (sparse)
# M: NxN diagonal lumped mass matrix (sparse)
```

Using this on grid-structured voxel coordinates is explicitly warned against. The README states the internal Delaunay triangulation "may not be totally robust to collinear or degenerate point clouds." (PyPI) Regular grids create exactly these degenerate configurations:

- **Perfect collinearity** along rows/columns causes undefined Delaunay circumcircles
- **Co-circularity** of grid cell corners makes triangulation non-unique (Wikipedia)
- **Axis-aligned tangent planes** can collapse projected neighborhoods
- An open GitHub issue reports `RuntimeError: GC_SAFETY_ASSERT FAILURE` (GitHub) (github) — precisely the expected failure mode on degenerate inputs

Even if it doesn't crash, the LBO eigendecomposition will suffer from **staircase artifacts**, **eigenvalue splitting/degeneracy from grid symmetries**, and **eigenvectors preferentially aligning with grid axes** rather than surface geometry.

The recommended path, in order of quality:

1. **Best: Marching cubes →** `mesh_laplacian(verts, faces)` — vertices land at sub-voxel interpolated positions (breaking the grid), producing a proper triangle mesh. Optionally add Taubin smoothing. This is the standard neuroimaging pipeline.
2. **Good: FPS or Poisson disk subsampling →** `point_cloud_laplacian()` — Farthest-point sampling produces blue-noise-distributed subsets that break all collinearity/co-circularity. Available in Open3D, PyTorch3D, trimesh.
3. **Adequate: Random jittering →** `point_cloud_laplacian()` — Adding uniform noise with magnitude ≪ voxel spacing breaks exact degeneracy, but the staircase approximation remains.
4. **Avoid: Raw voxel coordinates →** `point_cloud_laplacian()` — likely to crash or produce meaningless eigendecompositions.

Conclusion: integration assessment

BrainNet is **technically viable but practically challenging** to integrate today. The accuracy gains over alternatives are substantial (~**50% thickness error reduction**, Springer sub-millimeter surface distances), Springer +4 the speed is excellent (1 second GPU / minutes CPU), arXiv and the lack of FreeSurfer dependency is a real advantage. However, three factors complicate backend integration:

The absence of a public API is the biggest obstacle. You'll need to read the source, identify the model class and inference entry point, handle model weight downloading, and write your own wrapper. This is doable (it's 93% Python, Simnibs only 139 commits) but fragile — expect breaking changes with each release.

The dependency chain is heavy and brittle. PyTorch 2.6.0 + CUDA 12.4 + Kaolin 0.18.0 + manual CUDA extension compilation creates a narrow compatibility window. Kaolin in particular is notorious for version-pinning issues.

The project is very early-stage (4 GitHub stars, v0.2, no public documentation). There's real risk the API changes substantially before v1.0. For a library backend, you'd want to wrap it behind a stable interface and pin to a specific commit hash rather than tracking releases. The GPL-3.0 license GitHub GitHub also means your library would need to be GPL-compatible if you link to BrainNet directly.

For the `robust_laplacian` question: do not use `point_cloud_laplacian()` on raw voxel grids. Run marching cubes first (e.g., `skimage.measure.marching_cubes`), then use `mesh_laplacian(verts, faces)` for high-quality LBO eigendecomposition PyPI with none of the grid-degeneracy issues.